**Practical No. 4 : Implement a Sudoku solver using backtracking and constraint propagation techniques.**

```
def solve(b):
    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                for n in range(1, 10):
                    if (n not in b[r] and
                        all(b[i][c] != n for i in range(9)) and
                        all(b[i][j] != n
                            for i in range(r//3*3, r//3*3+3)
                            for j in range(c//3*3, c//3*3+3))):

                        b[r][c] = n
                        if solve(b):
                            return True
                        b[r][c] = 0
                return False
    return True


def print_board(b):
    for i in range(9):
        if i % 3 == 0 and i != 0:
            print("-" * 21)
        for j in range(9):
            if j % 3 == 0 and j != 0:
                print("|", end=" ")
            print(b[i][j], end=" ")
```

```
        print()

board = [
[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]
]

print("Original Sudoku:")
print_board(board)

if solve(board):
    print("\nSolved Sudoku:")
    print_board(board)
else:
    print("No solution exists.")
```